

System Design Document: Multi-Sensor Tracking & Fusion

1. System Architecture & Core Components

The system is designed with a strict Object-Oriented Programming (OOP) paradigm in Modern C++. It utilizes an **Event-Driven Architecture** to handle the asynchronous nature of the sensors (1Hz Radar, 10Hz EO). Instead of processing data in fixed time loops, the system maintains a unified chronological priority queue of all incoming measurements, processing them sequentially. The linear algebra and matrix operations are optimized using the **Eigen3** library.

The architecture is divided into the following highly decoupled modules:

- **Measurement (Data Layer):** A polymorphic base class holding the `timestamp` and `sensor_type`. Derived classes include `RadarMeasurement` (holds range, azimuth, elevation, radial velocity) and `EOMeasurement` (holds azimuth, elevation). This abstraction allows the core tracker to process heterogeneous data seamlessly.
- **EKF (Math Engine):** Encapsulates the purely mathematical operations of the Extended Kalman Filter. It holds the state vector `X` and the covariance matrix `P`. It provides methods for `predict(dt)` and specialized update functions (`updateRadar`, `updateEO`) which internally compute the respective Jacobians (`H`) to handle the non-linear transformations from Cartesian to Spherical coordinates.
- **Track (Entity Layer):** Represents a single tracked target in the real world. It owns an instance of the `EKF`. It manages the track's lifecycle attributes, including `track_id`, `age`, `coast_cycles` (consecutive missed associations), and its current state (`TENTATIVE`, `CONFIRMED`, `COASTING`).
- **DataAssociation (Logic Engine):** A stateless module responsible for matching incoming measurements to existing tracks. It utilizes the **Mahalanobis Distance** to construct a cost matrix and applies a Global Nearest Neighbor (GNN) strategy for assignment.
- **TrackerManager (The Brain):** The central orchestrator. It consumes the chronological measurement queue and executes the primary tracking loop: Predict -> Associate -> Update -> Track Management (Initiation/Deletion).

2. Assumptions & Simplifications

To maintain a robust baseline system, the following physical and statistical assumptions were made:

- **Kinematic Model:** The system assumes a **Constant Velocity (CV)** model in a 3D Cartesian coordinate system. Accelerations or deliberate maneuvers are treated as process noise (Q).
- **Measurement Noise:** Sensor errors are assumed to follow a zero-mean Gaussian distribution, accurately represented by the measurement covariance matrix R .
- **Clutter Distribution:** False alarms (clutter) are assumed to be uniformly distributed in the spatial domain and follow a Poisson distribution in the temporal domain.
- **Flat Earth:** For the scope of this generic 3D tracking scenario, Earth's curvature and Coriolis effects are deemed negligible.

3. Trade-offs & Design Decisions

- **Cartesian State Vector vs. Spherical State:**
 - *Decision:* The state vector is represented in 3D Cartesian coordinates $X = [x, y, z, v_x, v_y, v_z]$.
 - *Trade-off:* While measurements arrive in spherical coordinates (making a spherical state vector appealing for simple updates), targets in the real world move linearly in Cartesian space. A Cartesian state allows for a strictly linear and computationally cheap prediction step (F matrix), at the cost of requiring an Extended Kalman Filter (EKF) with non-linear Jacobians (H) for the update step.
- **Sequential Fusion vs. Centralized Batch Fusion:**
 - *Decision:* Measurements are fused sequentially as they arrive. If a Radar and an EO measurement share the exact same timestamp, the filter performs a **Predict** (with $dt=0$), updates with Radar, and then immediately updates the new state with EO.
 - *Trade-off:* Sequential updating is mathematically equivalent to batch updating (assuming uncorrelated sensor noise) but is significantly simpler to implement, requires no complex time-alignment buffering, and naturally solves the asynchronous rates (1Hz vs 10Hz).
- **Global Nearest Neighbor (GNN) vs. Multiple Hypothesis Tracking (MHT):**
 - *Decision:* Data association is handled using GNN with Mahalanobis gating.
 - *Trade-off:* GNN is highly computationally efficient and perfectly adequate for sparse to moderately cluttered environments. Implementing MHT or JPDA

would provide better track continuity in extreme crossing scenarios but introduces heavy computational overhead (exponential memory growth) and complexity outside the scope of a baseline real-time tracker.

- **Track Initiation Policy:**
 - *Decision:* New tracks are exclusively initiated by unassociated **Radar** measurements. Unassociated EO measurements are discarded.
 - *Trade-off:* EO sensors are passive (angles only) and lack range observability. Attempting to initialize a 3D Cartesian track from a single 2D ray introduces infinite depth uncertainty. Waiting for a Radar detection ensures the track is initialized with a bounded, fully observable 3D position.

4. Known Limitations & Failure Scenarios

The architectural choices deliberately accept the following limitations:

- **High-G Maneuvers:** Because the EKF relies on a linear Constant Velocity (CV) model, a target executing a sudden, high-G turn (e.g., a fighter jet) may step outside the Mahalanobis gating threshold before the covariance matrix (P) has time to inflate. This will result in track loss or track fragmentation. *Mitigation:* A future upgrade would employ an Interacting Multiple Model (IMM) filter to switch between CV and Constant Turn Rate (CTRV) models.
- **Sustained Close Proximity (Ghosting):** If two targets fly parallel to each other at the exact same velocity and within the same Radar resolution cell for a prolonged period, the GNN algorithm may exhibit “ID Switching” or mistakenly merge the two tracks.
- **Prolonged EO-Only Coasting:** If the Radar is jammed or drops out entirely, the system can sustain tracking using the 10Hz EO updates. However, because EO only observes angles, the uncertainty (Covariance P) along the depth/range axis will continuously inflate over time, eventually degrading the 3D accuracy.